

Proposta de Dominio - HubArena - Sprint 1

1. Dominio escolhido

O HubArena e uma plataforma distribuida voltada para reserva de quadras esportivas. O sistema conecta clientes interessados em reservar espacos esportivos com prestadores responsaveis por arenas, quadras e horarios disponiveis.

O dominio escolhido atende ao modelo cliente/prestador exigido pela disciplina, pois existe uma separacao clara entre quem consome o servico e quem administra a oferta. O cliente solicita reservas, enquanto o prestador cadastra arenas, gerencia quadras e aceita ou recusa solicitacoes.

2. Justificativa

A reserva de quadras esportivas ainda ocorre, em muitos casos, por mensagens, ligacoes telefonicas ou controles manuais. Esse processo pode gerar conflitos de horario, dificuldade de acompanhamento, baixa rastreabilidade e pouca transparencia para clientes e prestadores.

O HubArena propoe centralizar esse fluxo em uma aplicacao distribuida, com backend REST, persistencia em PostgreSQL e arquitetura preparada para comunicacao assincrona via RabbitMQ nas proximas sprints. O projeto e viavel para o escopo da disciplina porque permite evoluir progressivamente: primeiro com backend e banco de dados, depois com middleware orientado a mensagens e, por fim, com aplicativos moveis em Flutter.

3. Perfis de usuario

Cliente

O cliente e o usuario final interessado em reservar uma quadra esportiva. Suas principais acoes sao consultar arenas, visualizar quadras disponiveis, criar solicitacoes de reserva e acompanhar o status da reserva.

Prestador

O prestador e o administrador da arena esportiva. Suas principais acoes sao cadastrar arenas, cadastrar quadras, configurar dados das quadras, visualizar reservas e aceitar ou recusar solicitacoes.

4. Funcionalidades implementadas na Sprint 1

Na Sprint 1 foi implementado o backend REST do HubArena, com as operacoes essenciais do dominio.

As funcionalidades entregues foram:

- verificacao de funcionamento da API;
- verificacao da conexao com PostgreSQL;
- criacao e listagem de usuarios;
- consulta de usuario por ID;
- criacao e listagem de arenas;
- consulta de arena por ID;
- criacao e listagem de quadras;
- consulta de quadra por ID;
- criacao de reservas;
- listagem de reservas;
- consulta de reserva por ID;
- aceite de reservas pendentes;
- recusa de reservas pendentes.

No modelo do sistema, uma arena representa o estabelecimento esportivo. Uma quadra, representada pela entidade Court, pertence a uma arena e possui modalidade esportiva, preco por hora, capacidade e disponibilidade.

5. Arquitetura implementada

A arquitetura do projeto foi organizada como um repositorio geral chamado HubArena, preparado para as quatro sprints da disciplina.

A estrutura principal do repositorio e:

- backend: API REST em Flask;
- mobile/cliente: espaco reservado para o aplicativo Flutter do cliente;
- mobile/prestador: espaco reservado para o aplicativo Flutter do prestador;
- docs/sprint1: documentacao consolidada da Sprint 1;
- docs/sprint2, docs/sprint3 e docs/sprint4: pastas preparadas para as proximas entregas;
- postman: colecao Postman consolidada.

O backend foi implementado em Flask/Python e organizado em camadas:

- routes;
- controllers;
- services;
- repositories;
- models;
- database;
- config.

Essa separacao favorece organizacao, manutencao e aderencia a boas praticas de arquitetura em camadas. A persistencia foi realizada com PostgreSQL, executado via Docker Compose, e o acesso ao banco foi feito com SQLAlchemy.

HubArena - Proposta de Dominio - Sprint 1

O diagrama de arquitetura foi produzido em PlantUML e representa os componentes principais do sistema: App Cliente, App Prestador, Backend REST, PostgreSQL e RabbitMQ. Na Sprint 1, o RabbitMQ aparece como componente arquitetural planejado para a Sprint 2.

6. Endpoints REST implementados

A API REST implementada na Sprint 1 contem os seguintes grupos de endpoints:

Grupo	Metodo	Endpoint	Descricao
-------	--------	----------	-----------

--- --

Health	GET	/health	Verifica se a API esta ativa
Health	GET	/health/db	Verifica conexao com PostgreSQL
Users	POST	/users	Cria cliente ou prestador
Users	GET	/users	Lista usuarios
Users	GET	/users/{id}	Consulta usuario por ID
Arenas	POST	/arenas	Cria arena
Arenas	GET	/arenas	Lista arenas
Arenas	GET	/arenas/{id}	Consulta arena por ID
Courts	POST	/courts	Cria quadra
Courts	GET	/courts	Lista quadras
Courts	GET	/courts/{id}	Consulta quadra por ID
Reservations	POST	/reservations	Cria reserva
Reservations	GET	/reservations	Lista reservas
Reservations	GET	/reservations/{id}	Consulta reserva por ID
Reservations	PUT	/reservations/{id}/accept	Aceita reserva pendente
Reservations	PUT	/reservations/{id}/reject	Recusa reserva pendente

Esses endpoints cobrem o fluxo principal da Sprint 1: criacao de prestador, criacao de cliente, cadastro de arena, cadastro de quadra, criacao de reserva e atualizacao do status da reserva.

7. Tecnologias utilizadas

As tecnologias utilizadas na Sprint 1 foram:

- Python 3.12;
- Flask;
- Flask-SQLAlchemy;
- Flask-CORS;
- PostgreSQL;
- Docker Compose;
- pg8000;
- PlantUML;
- Postman;
- requests para testes automatizados.

A escolha por Flask permitiu construir uma API REST objetiva e compativel com o escopo da sprint. O PostgreSQL foi escolhido por oferecer persistencia relacional robusta e adequada ao crescimento futuro do sistema.

8. Artefatos produzidos

Foram produzidos os seguintes artefatos:

- backend REST funcional em Flask;
- banco PostgreSQL configurado via Docker Compose;
- schema documentado;
- diagrama de arquitetura em PlantUML;
- imagem PNG do diagrama de arquitetura;
- colecao Postman com endpoints da Sprint 1;
- README especifico da colecao Postman;
- relatorio de testes automatizados;
- README geral do projeto;
- README do backend;
- documento de proposta em Markdown e PDF.

9. Preparacao para as proximas sprints

A Sprint 1 preparou a base para evolucao do HubArena. Na Sprint 2, sera implementada a integracao com RabbitMQ, com publicacao e consumo de eventos como:

- reservation_created;
- reservation_accepted;
- reservation_rejected.

Na Sprint 3 sera desenvolvido o aplicativo Flutter do cliente. Na Sprint 4 sera desenvolvido o aplicativo Flutter do prestador e o fluxo completo de ponta a ponta.

10. Conclusao

O HubArena apresenta um dominio claro, viavel e alinhado aos requisitos da disciplina. A Sprint 1 estabelece a base tecnica do sistema distribuido

HubArena - Proposta de Dominio - Sprint 1

por meio de backend REST funcional, persistencia em PostgreSQL, documentacao arquitetural, colecao Postman e testes automatizados.

A solucao esta preparada para evoluir nas proximas sprints com comunicacao assincrona via RabbitMQ e desenvolvimento dos aplicativos moveis em Flutter.